

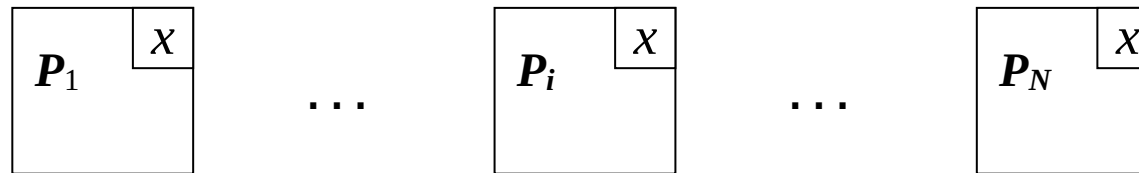


GŁOSOWANIE

– mechanizm pokonujący ograniczenia atomowego zatwierdzenia

Definicja problemu:

Należy zagwarantować atomową spójność operacji *read* i *write* na replikach obiektu x .



Reguła podstawowa:

Zanim proces P_i uzyska dostęp do obiektu x musi zdobyć odpowiednią liczbę głosów od pozostałych procesów (*quorum*)

Model systemu:

Każdy proces P_i (tj. każda replika) posiada pewną liczbę głosów V_i



STATYCZNE GŁOSOWANIE

Model systemu:

- każda operacja na replice wymaga uzyskania blokady:
 - wyłączonej (*exclusive-lock*)
 - współdzielonej (*shared-lock*)
- blokady zakłada lokalny zarządca (*lock manager*)
- każda replika posiada monotoniczny numer wersji równy ilości dokonanych modyfikacji VN_i
- model awarii *fail-recovery* (procesy i kanały)



Ogólna koncepcja:

- przed wykonaniem operacji zbieramy głosy od innych procesów, aż uzbieramy odpowiednie *quorum*
- operacja *read* wymaga *read-quorum* **R**
- operacja *write* wymaga *write-quorum* **W**

Problemy:

- kiedy zakończyć zbieranie głosów?
- którą kopię odczytać?
- które kopie uaktualniać?



Algorytm Gifforda

1. P_i składa *Lock_Request* do lokalnego zarządcy.
2. Gdy blokada została założona P_i wysyła komunikat *Vote_Request* do wszystkich procesów.
3. Gdy P_j odbiera *Vote_Request*, składa *Lock_Request* do lokalnego zarządcy. Jeśli blokada została założona, odsyła do P_i wersję swojej repliki VN_j oraz liczbę głosów V_j .
4. P_i decyduje o wykonaniu operacji w zależności od liczby uzbieranych głosów (*timeout*):

$V_{read} = \sum_{k \in \mathbb{O}} V_k$, gdzie $\mathbb{O}$ oznacza zbiór numerów procesów, które przysłały głosy

$V_{read} \geq R \Rightarrow P_i$ zebrał quorum dla odczytu

$V_{write} = \sum_{k \in \mathbb{Q}} V_k$, gdzie $\mathbb{Q} = \{ k \in \mathbb{O} : VN_k = VN_{max} \}$ i $VN_{max} = \max \{ VN_j : j \in \mathbb{O} \}$

$V_{write} \geq W \Rightarrow P_i$ zebrał quorum dla zapisu



5. Jeśli P_i nie zebrał quorum, wysyła *Release_Lock* do lokalnego zarządcy oraz do wszystkich $P_k : k \in \mathbb{Q}$.
6. Jeśli P_i zebrał quorum, sprawdza czy jego replika jest aktualna ($VN_i == VN_{max}$). Jeśli $VN_i < VN_{max}$, aktualna kopia jest sprowadzana od dowolnego $P_k : k \in \mathbb{Q}$.
7. Jeśli P_i ma wykonać operację *read*, czyta lokalną kopię. Jeśli *write* – modyfikuje lokalną kopię oraz VN_i i wysyła uaktualnienie do wszystkich $P_k : k \in \mathbb{Q}$ (uaktualnienie wyłącznie świeżych replik). Wówczas P_i wysyła *Release_Lock* do lokalnego zarządcy oraz do wszystkich $P_k : k \in \mathbb{Q}$.
8. Każdy P_j otrzymując uaktualnienie modyfikuje lokalną replikę, a otrzymując *Release_Lock* zwalnia blokadę.

Problemy:

- Jak zagwarantować poprawność **R** i **W** ?
- Jak rozdzielić głosy pomiędzy procesy?



Jak zagwarantować poprawność **R** i **W**?

- $V = \sum_i V_i$
- *majority* $M = \left\lceil \frac{V+1}{2} \right\rceil$
- **W** $\geq M$
- **R** + **W** $> V$

To gwarantuje, że:

- zawsze istnieje podzbiór aktualnych replik dysponujących sumą **W** głosów
- **W** jest odpowiednio duże by wykluczyć współbieżne modyfikacje w rozłącznych podzbiorach replik
- część wspólna *read-quorum* i *write-quorum* nie jest pusta, dzięki czemu w każdym *read-quorum* będzie co najmniej jedna aktualna replika



Jak rozdzielić głosy pomiędzy procesy?

$$V = \sum_i V_i = 5$$

2 $V_2 = 1$
speed = 2
reliability = 3

1 $V_1 = 1$
speed = 2
reliability = 3

3 $V_3 = 2$
speed = 8
reliability = 3

4 $V_4 = 1$
speed = 5
reliability = 8

(a) $M = 3$, **$W = 4$** , **$R = 2$**

- szybkie odczyty (**R** małe) – patrz: P_3
- awaria procesu P_3 uniemożliwia zebranie *write-quorum*



$$V = \sum_i V_i = 5$$

2 $V_2 = 1$
speed = 2
reliability = 3

(b) $M = 3$, **$W = 3$** , **$R = 3$**

1 $V_1 = 1$
speed = 2
reliability = 3

3 $V_3 = 2$
speed = 8
reliability = 3

4 $V_4 = 1$
speed = 5
reliability = 8

- zebranie *read-quorum* lub *write-quorum* wymaga 3 dostępnych jednocześnie procesów lub P_3 i przynajmniej jednego innego procesu



- akcent na niezawodność (proces P_4)

$$V = \sum_i V_i = 5$$

2 $V_2 = 1$
speed = 2
reliability = 3

(b) $M = 3$, **$W = 3$** , **$R = 3$**

1 $V_1 = 1$
speed = 2
reliability = 3

3 $V_3 = 1$
speed = 8
reliability = 3

4 $V_4 = 2$
speed = 5
reliability = 8



Uwagi:

- Nie trzeba zliczać wyłącznie głosów z VN_{max} aby uzyskać *write-quorum*.
- Starsze repliki mogą być uaktualniane przy okazji wykonywania operacji *write*.

Dlaczego głosowanie nazywa się statyczne?

- V_i , **R** oraz **W** są stałe i niezależne od bieżącego stanu systemu

Problem:

- co jeśli P_3 i jednak P_4 jednocześnie ulegną awarii?
- lub oddzielią się od pozostałych (rozłączenie sieci komunikacyjnej)?



Partycjonowanie:

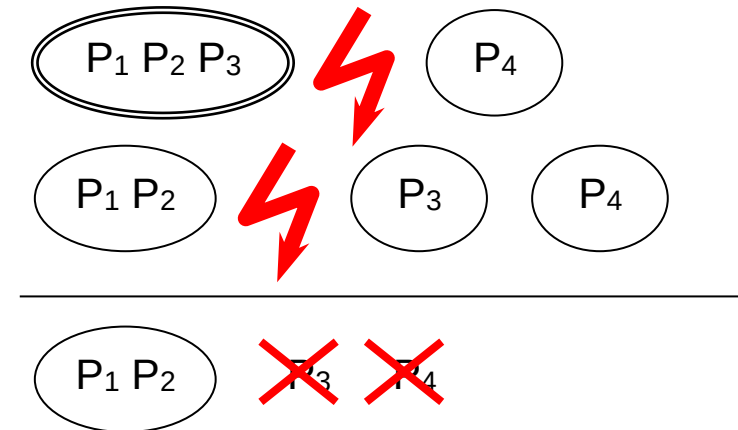
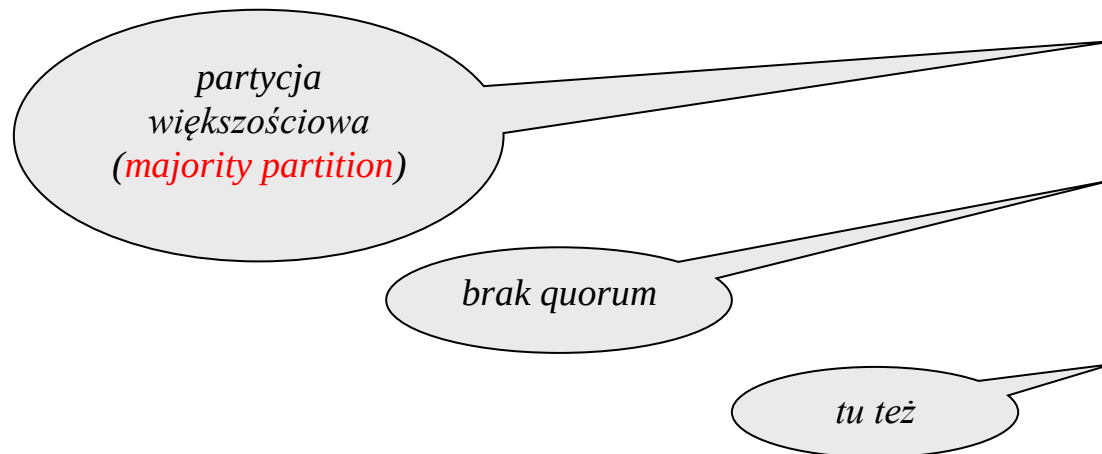
2 $V_2 = 1$

$M = 3, W = 3, R = 3$

1 $V_1 = 1$

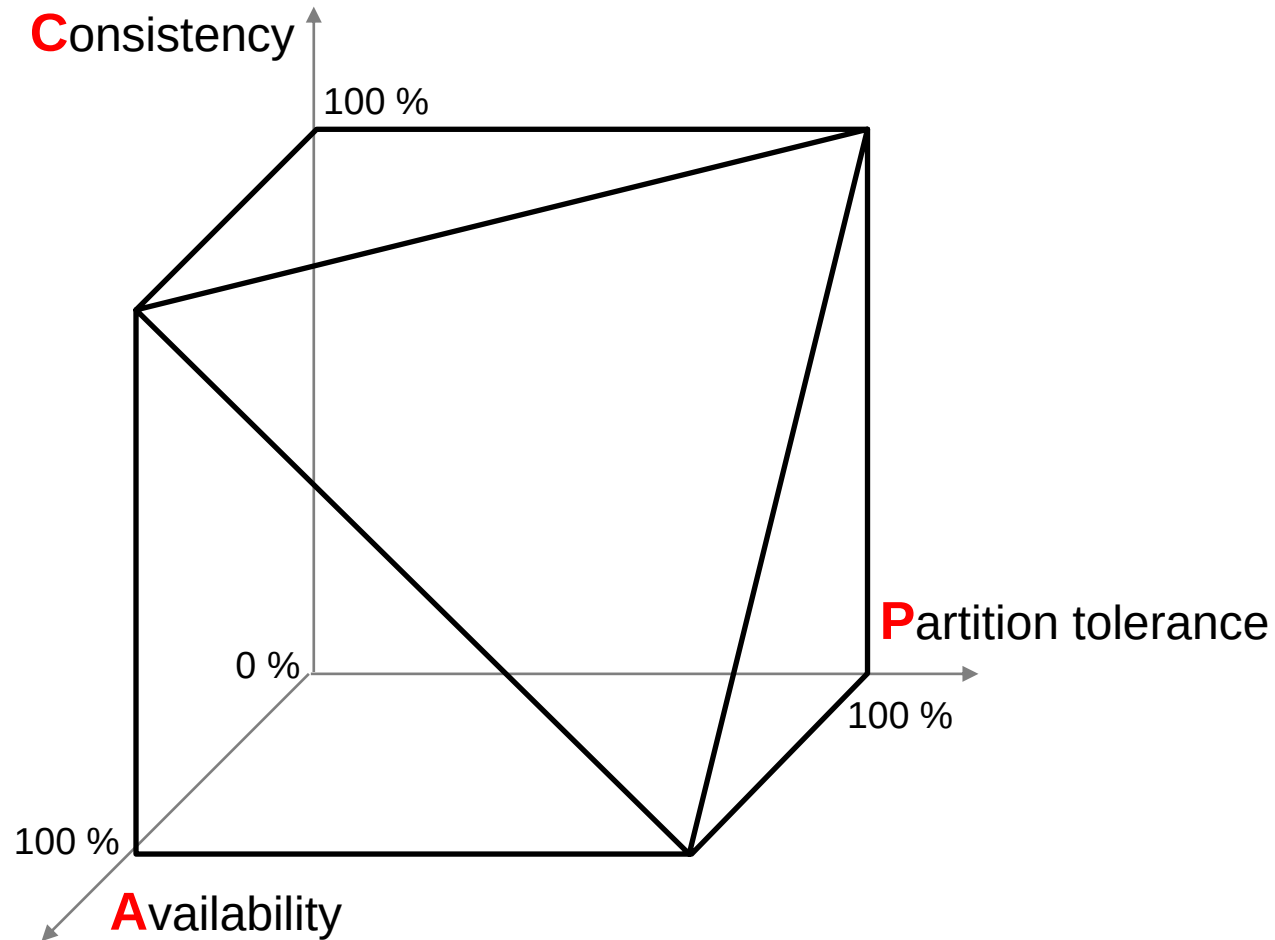
3 $V_3 = 1$

4 $V_4 = 2$





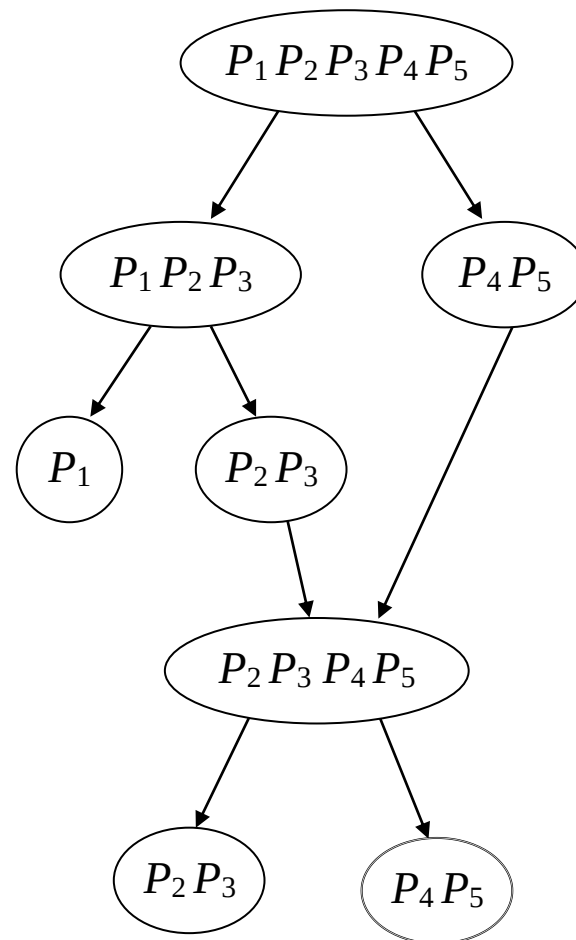
CAP IMPOSSIBILITY RESULT





Partitioning graph:

- reprezentuje historię podziału sieci: wierzchołki = partycje, krawędzie = podział lub scalenie





DYNAMICZNE GŁOSOWANIE

Idea:

- adaptować liczbę głosów lub zbiór głosujących procesów do stanu systemu po awarii, tak aby móc zebrać quorum

Warianty:

- *majority based voting* – zmienny zbiór procesów stanowiących większość niezbędną do modyfikacji replik
- *dynamic vote reassignment* – zmienna liczba głosów przypisanych do poszczególnych replik



Majority-based dynamic voting protocol

- the set of sites that form a majority is dynamically altered to enhance availability in the event of failure
- those are sites that were updated when the most recent update was performed

Voting protocol (Jajodia-Mutchler [1])

- selects one of the partitions where read and write operations can continue – the one that could have formed a majority in the configuration of the last update
- if only it is possible to distinguish a *primary partition*



Assumptions:

- for simplicity, each process holds one distinct replica with a single vote
- all processes are linearly numbered $P_1, P_2, \dots, P_i, \dots$

Protocol data:

- **Version Number** VN_i – counts the number of successful updates of the replica at P_i
- number RU_i of **Replicas Updated** in the most recent update
- **Distinguished Sites** list DS_i depending on RU_i – when RU_i is even, DS_i identifies the replica that is greater (in linear ordering) than all the other replicas participating in the most recent update of the replica at P_i . When RU_i is odd $DS_i = \emptyset$ *)

* actually except for $RU_i=3$, in which case the original protocol switches to static voting and DS_i lists the three replicas that participated in the most recent update – Jajodia and Mutchler found that static voting performs better than dynamic voting when the number of replicas is 3 – however, we omit this here



Current state:

	P_1	P_2	P_3	P_4	P_5
VN	3	3	3	3	3
RU	5	5	5	5	5
DS	—	—	—	—	—

each replica has
already been updated
3 times so far

all the sites are
accessible to the
others

irrelevant

P_2 wants to perform a write

and finds that it can communicate only with P_1 and P_3



primary
partition

since $VN_{max} = 3$ and RU associated with VN_{max} is 5

and partition $P_1 P_2 P_3$ has 3 of the 5 copies

P_2 knows that it belongs to the distinguished primary partition



	P_1	P_2	P_3	P_4	P_5
VN	4	4	4	3	3
RU	3	3	3	5	5
DS	—	—	—	—	—

P_3 wants to perform a write

and finds that it can communicate only with P_2



P_3 knows the latest version is $VN_{max} = 4$ and $RU = 3$

and since partition P_2P_3 has 2 of the 3 copies, P_3 belongs to the primary partition thus processes the write



since the new majority partition $P_2 P_3$ has 2 sites, $RU=2$ is even,
so DS is set to P_2 (because P_2 has the highest order from $\{P_2, P_3\}$)

	P_1	P_2	P_3	P_4	P_5
<i>VN</i>	4	5	5	3	3
<i>RU</i>	3	2	2	5	5
<i>DS</i>	—	P_2	P_2	—	—

Now, P_4 wants to perform a write

and discovers that it can communicate with P_2, P_3 and P_5



the latest version in $P_2 P_3 P_4 P_5$ is $VN_{max} = 5$ with $RU = 2$ and both $P_2 P_3$
belong to the new partition which becomes the primary partition

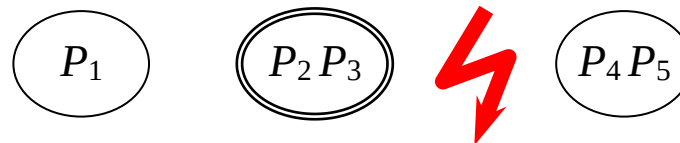


$RU=4$ is even, so DS is set to P_2 (P_2 still has the highest order):

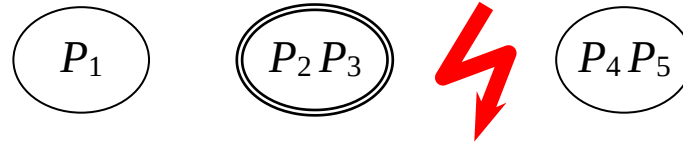
	P_1	P_2	P_3	P_4	P_5
VN	4	6	6	6	6
RU	3	4	4	4	4
DS	—	P_2	P_2	P_2	P_2

Finally, P_3 wants to perform a write

and discovers that it can communicate with B



since partition P_2P_3 contains exactly half the sites from the former partition and contains the distinguished site P_2 (DS is used to break the tie), the update is carried out in (primary) partition P_2P_3

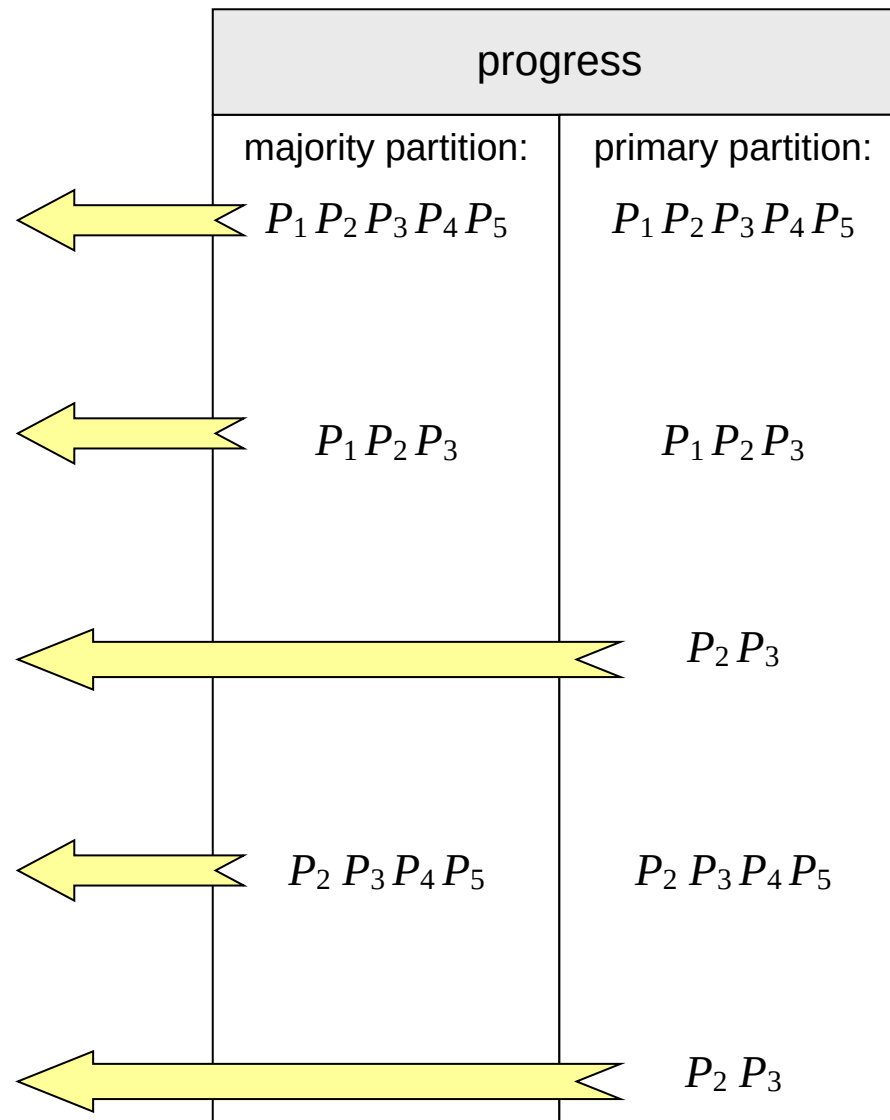
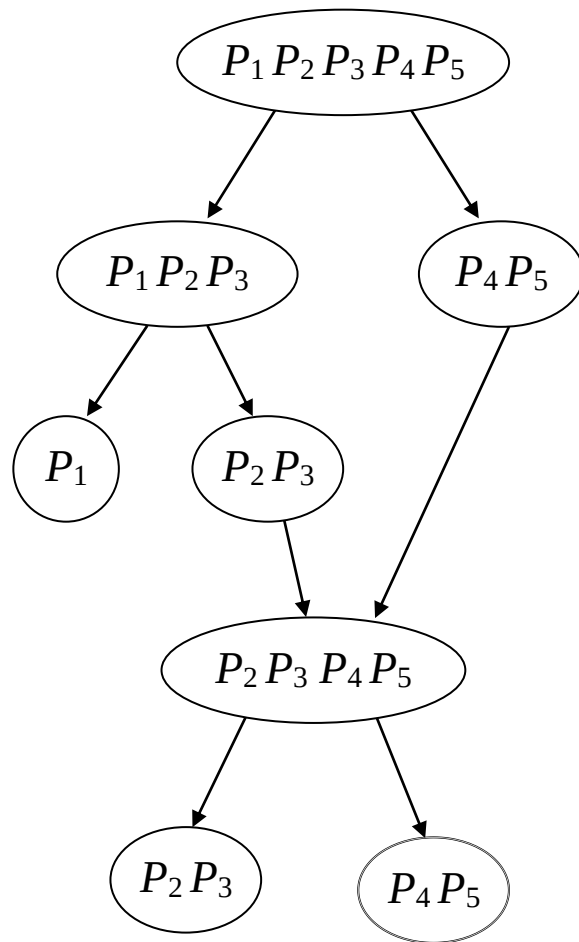


... the update is carried out in (primary) partition $P_2 P_3$:

	P_1	P_2	P_3	P_4	P_5
VN	4	7	7	6	6
RU	3	2	2	4	4
DS	—	P_2	P_2	P_2	P_2

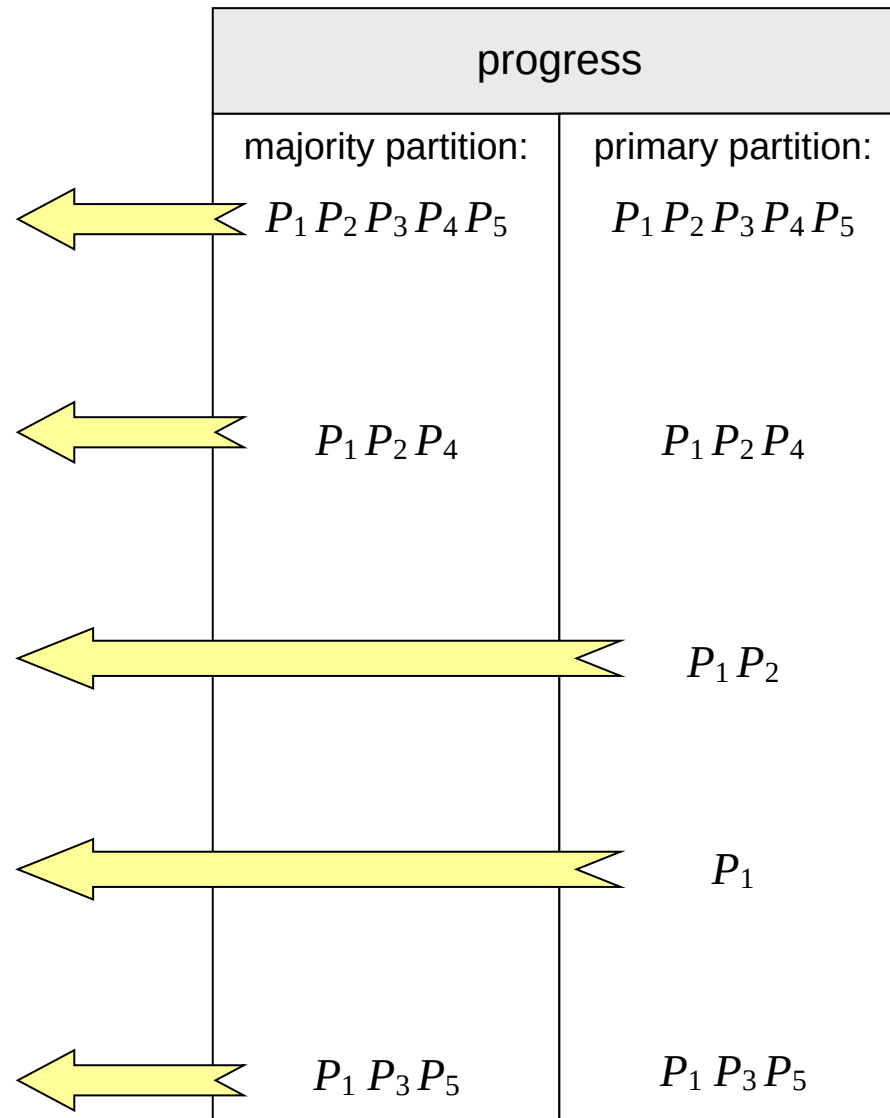
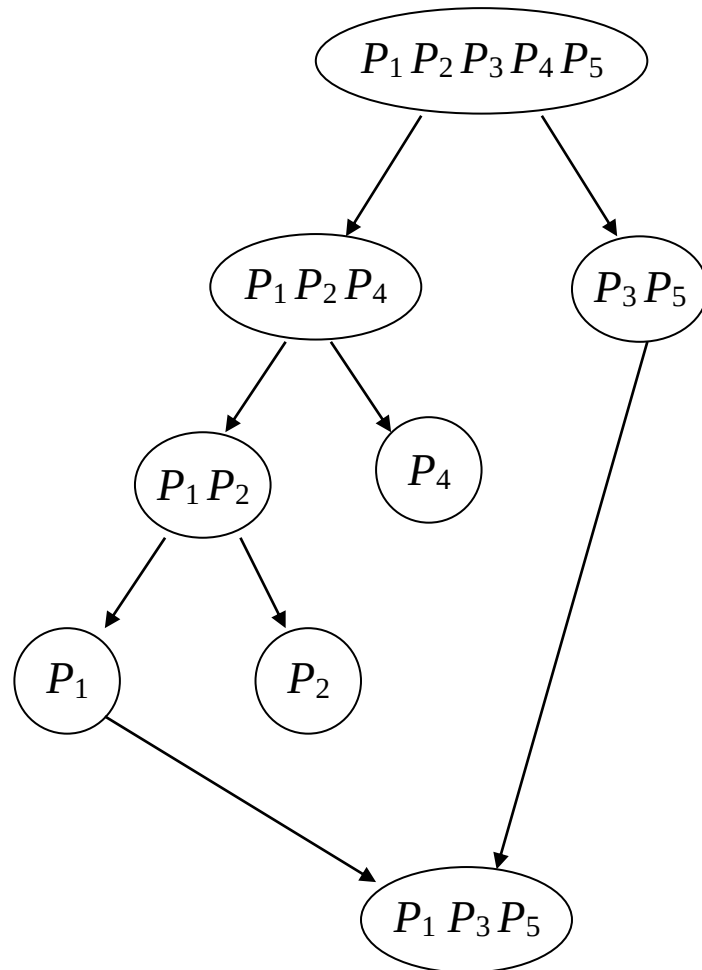


Static vs. dynamic voting:



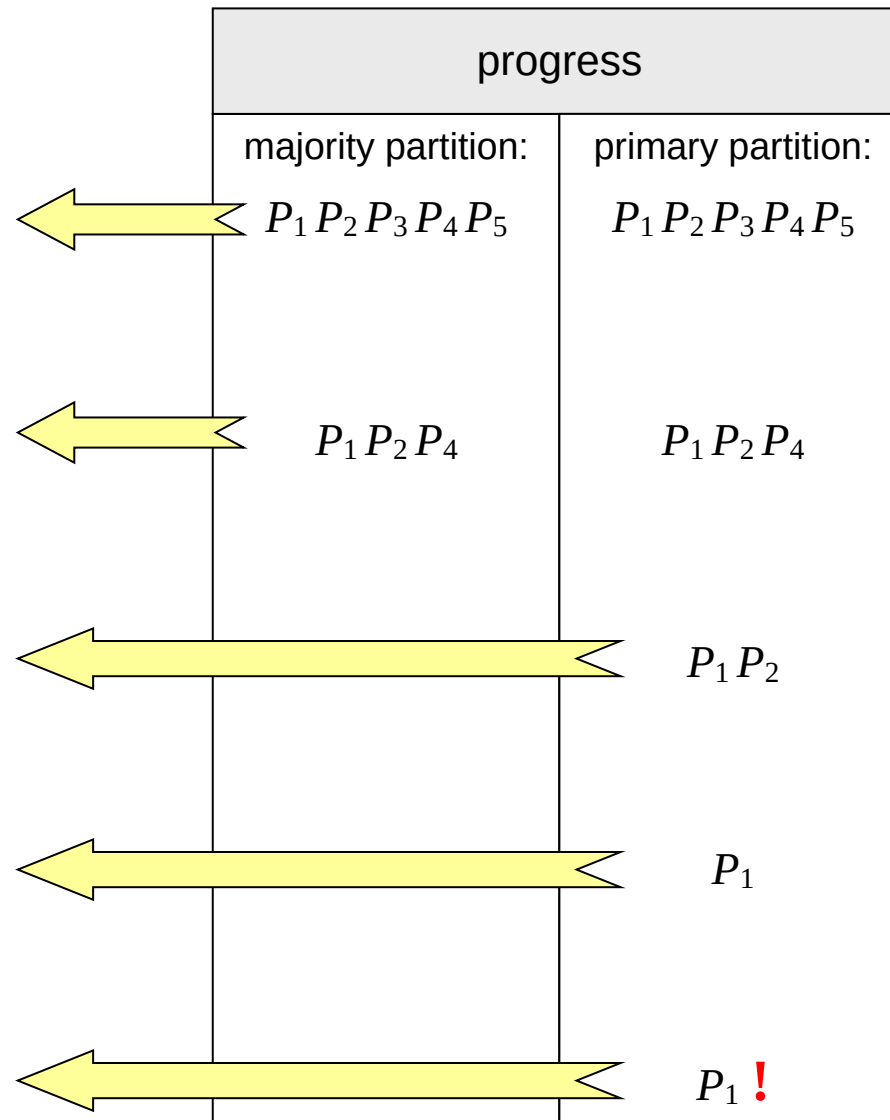
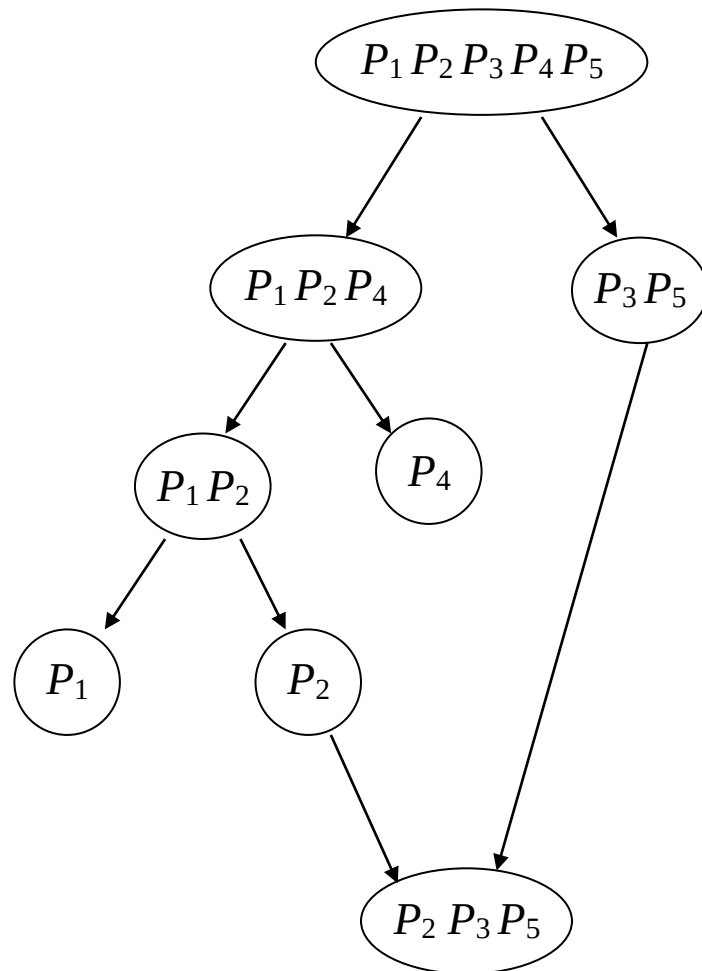


Static vs. dynamic voting:





Static vs. dynamic voting:





Dynamic vote reassignment

Strategies:

Group Consensus

- the sites in the active (primary) group agree upon the new vote assignment using consensus algorithm
- ☹ consensus is quite complicated

Autonomous Reassignment

- each site uses a view of the system to make a decision about changing its votes and picking a new V_i without regarding the rest of the sites
- ☹ may not be optimal but ...
- 😊 ... is quick, simple and flexible



Autonomous Reassignment

Vote increasing policies:

1. Overthrow technique

- one site in the active group takes on more votes
- election required

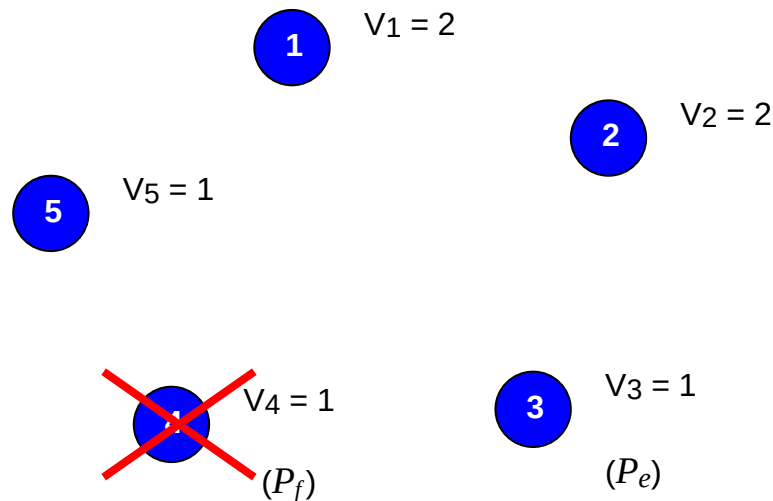
2. Alliance technique

- all the sites in the active group increase their votes



1. Overthrow technique

- total number of votes: $V = \sum_i V_i$
- majority: $M = \left\lceil \frac{V+1}{2} \right\rceil = \mathbf{W}$
- P_f has failed
- P_e has been elected to increase its voting power by $2V_f$
- $V' = V + V_f$ $\mathbf{W}' = \mathbf{W} + V_f$





2. Alliance technique

- e.g. all n remaining sites increase their voting power by $\left\lceil \frac{2V_f}{n} \right\rceil$ each
- or all n remaining sites increase their voting power by $2V_f$ each
- **W'** changes accordingly



LITERATURA

- [1]  S. Jajodia, D. Mutchler, “Dynamic Voting”, ACM SIGMOD Conference, 1987
- [2]  A. Schiper, “Replication. Theory and Practice”, Springer, 2010